Billing, Usage & Credits

Subscriptions, usage metering, credit ledger, UI flows and contracts

1. Subscriptions

HLD / responsibilities

Billing owns subscription terms, invoice generation, renewal scheduling, trials, grace periods and dunning state. Payment execution is delegated to Payments. Entitlement policy consumes subscription lifecycle events.

LLD and lifecycle

- Model subscription state separately from payment state: incomplete, trialing, active, past_due, paused, cancelled, expired, suspended as supported.
- Record currentPeriodStart/currentPeriodEnd and nextBillingAt. Use a scheduler that claims due work with a lease or atomic status update.
- Each billing period creates one invoice using a unique tenant + subscription + period key.
- Retry jobs must be replay-safe. A failed provider call may have succeeded; reconcile before creating another attempt.
- Cancellation request and effective cancellation are separate. cancelAtPeriodEnd=true preserves service until the period boundary unless policy says otherwise.
- Failed renewals follow configured retry cadence and grace period; entitlement policy is explicit and product-specific.
- Proration, plan migration and taxes should be feature-gated until tested.

Schema and DTOs

```
type SubscriptionDoc = {
  _id:ObjectId; tenantId:ObjectId; publicId:string; customerId:ObjectId;
  productId:ObjectId; priceId:ObjectId; status:string;
  currentPeriodStart:Date; currentPeriodEnd:Date; nextBillingAt?:Date;
  cancelAtPeriodEnd:boolean; cancelledAt?:Date; trialEndsAt?:Date;
  version:number; createdAt:Date; updatedAt:Date;
};
// Index {tenantId, status, nextBillingAt}; unique {tenantId, publicId}

export type CreateSubscriptionDTO = {
  customerId:string; priceId:string; trialDays?:number;
  paymentMethodReference?:string; metadata?:Record<string,unknown>;
};
export type SubscriptionDTO = {
  id:string; customerId:string; productId:string; priceId:string; status:string;
  currentPeriodStart:string; currentPeriodEnd:string; nextBillingAt?:string;
  cancelAtPeriodEnd:boolean; version:number;
};
export type CancelSubscriptionDTO = { timing:"immediately"|"period_end"; reason?:string };
```

Endpoints

Route	Contract
POST /api/v1/subscriptions	201 SubscriptionDTO or 202 pending setup; validate recurring price.
GET /api/v1/subscriptions/:id	200 SubscriptionDTO.
POST /api/v1/subscriptions/:id/cancel	CancelSubscriptionDTO; 200 updated subscription; 409 invalid state.
POST /api/v1/subscriptions/:id/resume	Allowed only for eligible scheduled cancellation/paused state.
GET /api/v1/subscriptions/:id/invoices	List related invoices.

UI

Screen	Components / behavior
Plans	PlanCards, BillingIntervalToggle, PriceDisclosure; server catalog authoritative.
Subscription details	StatusTimeline, PeriodDates, PaymentMethodSummary, Cancel/Resume actions.
Cancel flow	Reason select, effective-date confirmation, explicit impact on access.
Billing history	InvoiceTable, invoice detail, payment attempt and retry status.
Dunning operations	PastDueQueue, RetrySchedule, manual retry guarded by idempotency.

Sequence

```
sequenceDiagram
  participant UI as Plans UI
  participant SDK as kubera.billing
  participant API as Express
  participant B as BillingService
  participant DB as MongoDB
  participant Q as Renewal Worker
  participant P as Payments
  UI->>SDK: subscriptions.create(dto)
  SDK->>API: POST /subscriptions
  API->>B: validate price and create subscription
  B->>DB: Save subscription + initial invoice/outbox
  API-->>UI: 201/202 SubscriptionDTO
  Q->>DB: Claim due renewal
  Q->>B: finalize invoice for period
  B->>P: create payment attempt
  P-->>B: payment state
  B->>DB: update invoice/subscription + outbox
```

2. Usage metering

LLD

- Accept usage events with unique eventId per tenant; persist before returning accepted.
- Validate metric, unit, quantity bounds, timestamp window and customer entitlement to use the meter.
- Aggregation is explicit per price: sum, max, last or an approved tier calculation. Define timezone/billing-period boundaries.
- Late events use a configurable policy: accept within correction window, create adjustments, or reject with a reason.
- Finalized invoices are not silently recalculated. Corrections create adjustment/credit-note workflows.
- Use partitioning/retention/archive strategy when event volume becomes large; avoid unbounded arrays in customer documents.

Schema and DTO

```
type UsageEventDoc = {
  _id:ObjectId; tenantId:ObjectId; customerId:ObjectId; eventId:string;
  metric:string; quantity:number; unit:string; occurredAt:Date; receivedAt:Date;
  idempotencyKey?:string; metadata:Record<string,unknown>;
};
// Unique {tenantId, eventId}; index {tenantId, customerId, metric, occurredAt}

export type CreateUsageEventDTO = {
  customerId:string; eventId:string; metric:string; quantity:number;
  unit:string; occurredAt:string; metadata?:Record<string,unknown>;
};
export type UsageSummaryDTO = {
  customerId:string; metric:string; periodStart:string; periodEnd:string;
  quantity:number; unit:string; status:"open"|"finalized"|"adjusted";
};
```

Endpoints and UI

Route	Contract
POST /api/v1/usage/events	202 accepted; repeated event returns same receipt; conflicting payload for same event ID returns 409.

Route	Contract
GET /api/v1/usage/summary	Customer, metric and period filters; paginated.
GET /api/v1/usage/events/:eventId	Return accepted event status and billing association.
POST /api/v1/usage/adjustments	Privileged adjustment, reason and audit required.

UI: Usage Overview -> select customer/metric/period -> consumption chart and event table -> inspect event -> view invoice association -> create approved adjustment. Chart data is server state; filters belong in URL; adjustment form is local state.

Sequence

```
sequenceDiagram
  participant App as CPG Product
  participant SDK as kubera.usage
  participant API as Express
  participant U as UsageService
  participant DB as MongoDB
  participant Q as Aggregation Worker
  App->>SDK: usage.record(dto)
  SDK->>API: POST /usage/events
  API->>U: validate tenant, metric, idempotency
  U->>DB: Insert usage event
  API-->>App: 202 accepted
  Q->>DB: Read unaggregated events for period
  Q->>DB: Upsert aggregate keyed by customer+metric+period
  Q->>DB: Link aggregate to invoice when finalized
```

3. Credit ledger

Rules

- Represent grants, consumption, expiry, refunds and adjustments as append-only entries.
- Balance is a projection of entries, optionally materialized for performance; ledger remains source of truth.
- Consumption uses atomic conditional balance reservation or a transaction/serialized account lock.
- Credits may have buckets with source, expiry and spend priority (e.g. expiring credits first); make ordering deterministic.
- Concurrent consumption cannot overspend unless tenant policy explicitly enables overdraft.
- Credit grant from purchase is idempotent on source order/invoice/line.

Schema / API

```
type CreditEntryDoc = {
  _id:ObjectId; tenantId:ObjectId; customerId:ObjectId; accountId:string;
  entryType:"grant"|"consume"|"expire"|"refund"|"adjustment"|"reversal";
  quantity:number; sourceType:string; sourceId:string; expiresAt?:Date; createdAt:Date;
};
// Unique {tenantId, sourceType, sourceId, entryType, accountId} where policy allows one entry.

export type ConsumeCreditsDTO = { customerId:string; accountId:string; quantity:number; referenceId:string };
export type CreditBalanceDTO = { customerId:string; accountId:string; available:number; reserved:number; unit:string; asOf:string }
```

Route	Contract
GET /api/v1/credits/balance	200 CreditBalanceDTO.
POST /api/v1/credits/consume	Idempotency-Key required; atomic; 201 consumption receipt or 409 insufficient balance.
POST /api/v1/credits/adjustments	Privileged grant/adjustment with reason and audit.
GET /api/v1/credits/entries	Cursor-paginated ledger entries.

UI and sequence

UI: Credit Account -> current available balance -> grants/consumption/expiry timeline -> adjustment dialog restricted to finance/admin. Display a clear distinction between available, reserved and expired quantities.

```
sequenceDiagram
  participant App as CPG Product
```

```
participant SDK as kubera.credits
participant API as Express
participant C as CreditService
participant DB as MongoDB
App->>SDK: credits.consume(dto)
SDK->>API: POST /credits/consume + Idempotency-Key
API->>C: consume(tenant, dto)
C->>DB: Atomic balance condition / transaction
alt sufficient balance
  C->>DB: Append consumption entry and update projection
  API-->>App: 201 consumption receipt
else insufficient balance
  API-->>App: 409 INSUFFICIENT_CREDITS
end
```